

Lab 5 - Cloud Data Engineering Pipeline

Group 3 · Healthcare admissions dataset (Kaggle: [prasad22/healthcare-dataset](#)) · BigQuery sandbox, project lab-5-healthcare

We designed and built our cloud pipeline for the healthcare admissions dataset following the **source** → **load/validate** → **warehouse** → **serve pattern**. Below we walk through each stage in order, showing what we designed on paper and, where we went further, what we actually implemented and ran.

1. Source - healthcare CSV into the BigQuery sandbox

We uploaded our healthcare_dataset.csv directly into a new BigQuery sandbox project (lab-5-healthcare), inside a dataset called healthcare_lab_5, as a table named Admissions. This created a real table of 55,500 rows (8.06 MB), matching our dataset exactly.

Cost note: Storage is billed by GB per month, cents at this scale (~8 MB), negligible even kept indefinitely.

2. Load + Validate

On paper, this stage is an automated task (a Cloud Function or Composer job) that checks the incoming data before it is trusted for example, rejecting rows with a negative Billing Amount, the same rule we built in Lab 3's ingest.py.

We did not stop at the paper design: we rebuilt this logic as a real, working BigQuery pipeline (BigQuery Studio's Pipelines feature, built on Dataform), chaining two stages so each one reads from the previous stage's output rather than from raw data independently:

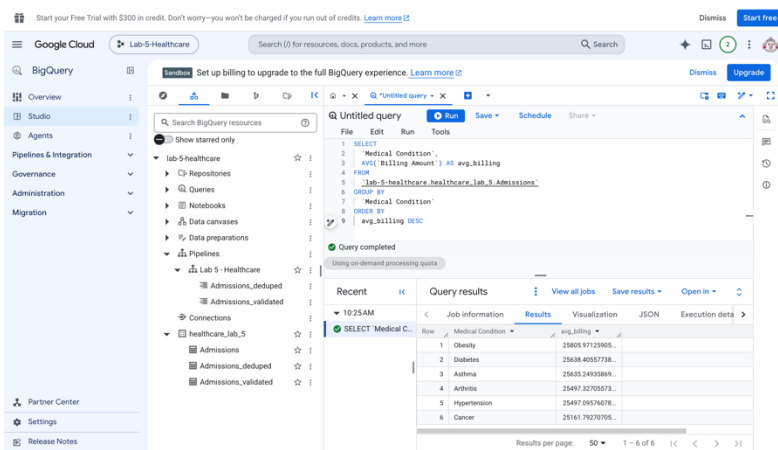


Admissions (55,500, raw) → Admissions_validated (55,392, billing ≥ 0) → Admissions_deduped (54,860, duplicates removed) - row counts match Lab 3's ingest.py exactly.

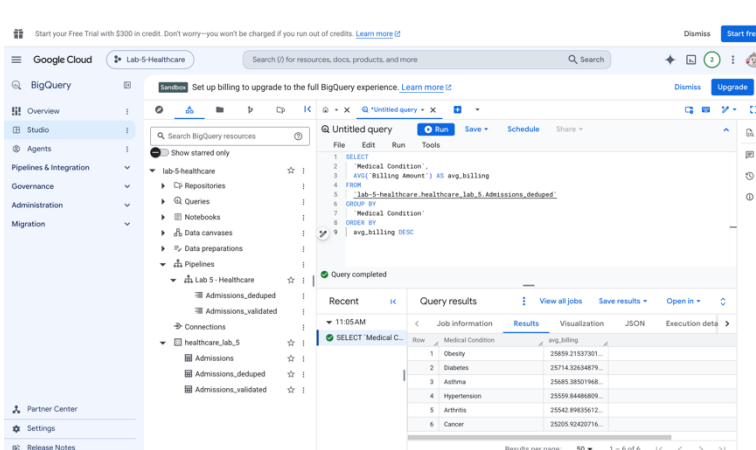
Cost note: Each stage only costs money while actively running, billed per second, and runs briefly and infrequently. Scheduling this pipeline to run automatically (rather than manually, as we did) would require billing to be enabled, the same sandbox restriction.

3. BigQuery - querying raw vs. cleaned data

We ran our analytical query, average billing amount by medical condition, first against the **raw Admissions table**, then again against the final, **cleaned Admissions_deduped table**, to see whether validation and deduplication actually changed the answer.



On raw Admissions (55,500 rows)



On cleaned Admissions_deduped (54,860 rows)

Medical Condition	Raw avg billing	Cleaned avg billing	Change
Obesity	25,805.97	25,859.22	+53.25
Diabetes	25,638.41	25,714.33	+75.92
Asthma	25,635.25	25,685.39	+50.14
Hypertension	25,497.10	25,559.84	+62.75
Arthritis	25,497.33	25,542.90	+45.57
Cancer	25,161.79	25,205.92	+44.13

Every average rose slightly once negative-billing rows and exact duplicates were removed and Hypertension and Arthritis swapped rank order (Arthritis led narrowly on raw data; Hypertension leads after cleaning). This shows the validation step is not just a formality: it can change which condition ranks where, which matters if this number were used to **prioritize resources**.

Cost note: Partitioning can cut scan costs ~100x for date-bound queries at scale. We could not demonstrate this live: our dataset's Date of Admission values span 2019–2024, and the BigQuery sandbox automatically deletes any partition older than 60 days regardless of when the table itself was created. Since every admission date in our data is far older than 60 days, the moment we partitioned by that column, every partition was deleted instantly, a genuine sandbox limitation that would need billing enabled in production.

4. Serve

Finally, this cleaned table feeds a dashboard for hospital administrators, and feature tables for predictive models for example, estimating length of stay or billing risk. We chose to build this as a Streamlit dashboard, since it is Python-based and lets us reuse the same pandas/BigQuery skills from our earlier labs: a small Python app queries Admissions_deduped directly and renders the results as interactive charts.

Cost note: The serving layer only reads small, already-summarized results, not the full raw dataset. Since egress (data leaving the cloud) is billed per gigabyte, keeping what leaves small keeps this cost low too. Streamlit itself is free to run locally or host on Streamlit Community Cloud, adding no extra cost to this stage.